Computer Security and privacy
Prof. Thomas Bourgeat

Luigi Ieracitano

October 10, 2026

Contents

1 Introduction to Computer Security **3**

1.1 Why a course on computer security? . . . 3
1.2 What is computer security? . . . 3
1.2.1 Traditional properties: the CIA triad . . . 3
1.2.2 Not-so-traditional (but important) properties . . . 4
1.3 The security policy . . . 4
1.4 The adversary and the threat model . . . 4
1.4.1 Key vocabulary to talk about the adversary . . . 5
1.5 Securing a system: security mechanisms . . . 5
1.5.1 Composing multiple mechanisms . . . 6
1.5.2 Humans are part of the system . . . 6
1.6 How do we show systems are secure? . . . 6
1.7 Security engineering . . . 7
1.8 Summary . . . 7

2 Principles of Computer Security **8**

2.1 Motivation . . . 8
2.1.1 Motivating example: the Heartbleed bug . . . 8
2.2 The eight core principles . . . 9
2.2.1 1 – Economy of mechanism . . . 9
2.2.2 2 – Fail-safe defaults . . . 9
2.2.3 3 – Complete mediation . . . 10
2.2.4 4 – Open design . . . 10
2.2.5 5 – Separation of privilege . . . 11
2.2.6 6 – Least privilege . . . 12
2.2.7 7 – Least common mechanism . . . 12
2.2.8 8 – Psychological acceptability . . . 13
2.3 Two extra principles (from physical security) . . . 13
2.3.1 9 – Work factor . . . 13
2.3.2 10 – Compromise recording . . . 14
2.4 Applying the principles: a machine learning example . . . 14
2.5 Summary . . . 14

3 Access Control **15**

3.1 Introduction to access control . . . 15
3.1.1 What is access control? . . . 15
3.1.2 Why is access control so important? . . . 16

3.1.3 Where does access control fit? Authentication vs. authorization 16
3.1.4 Implementing access control: the reference monitor 16
3.1.5 Who decides the access policy? DAC vs. MAC 17
3.2 Discretionary Access Control (DAC) 17
3.2.1 Historical motivation: Lampson's protection 17
3.2.2 The Access Control Matrix 18
3.2.3 Access Control Lists (ACLs) 19
3.2.4 Role Based Access Control (RBAC) 19
3.2.5 Group Based Access Control 20
3.2.6 Capabilities 20
3.3 Ambient authority and the confused deputy 21
3.3.1 Ambient authority 21
3.3.2 The confused deputy problem 21
3.4 Summary of the DAC concepts 22
3.5 DAC in practice: Linux (UNIX) and Windows 22
3.5.1 UNIX: principals and groups 22
3.5.2 Windows 26
3.5.3 Comparison UNIX vs. Windows 27
3.6 Summary 27

Chapter 1
Introduction to Computer Security

1.1 Why a course on computer security?

When designing "classical" systems or programs, we usually aim to guarantee three properties:

- **Correctness**: for a given input, the program provides the expected output.
- **Safety**: well-formed programs cannot produce bad (even dangerous) outputs.
- **Robustness**: the system copes with errors, both in the input and during execution.

In all these cases, we try to anticipate what could go wrong and take it into account in the design. Computer security differs from these disciplines by one extra ingredient: the presence of an **adversary**, i.e., an entity that *actively and intelligently* tries to make the system fail, as opposed to a random fault or accidental error.

1.2 What is computer security?

Definition 1 (Computer security)**.** Properties of a computer system must hold **in the presence of a resourced strategic adversary**.

Two elements are therefore central to any security analysis:

1. **Which properties** must be guaranteed?
2. **Against which adversary** (with which capabilities) must they be guaranteed?

1.2.1 Traditional properties: the CIA triad

Definition 2 (Confidentiality)**.** Prevention of unauthorized disclosure of information.
Example: the adversary should not be able to read my bank statement.

Definition 3 (Integrity)**.** Prevention of unauthorized modification of information.
Example: the adversary should not be able to change my bank balance.

Definition 4 (Availability)**.** Prevention of unauthorized denial of service.
Example: the adversary should not be able to prevent me from accessing my bank account.

These three properties form the **CIA triad** (*Confidentiality, Integrity, Availability*), which must be known by heart: it comes up in almost every exercise in this course.

1.2.2 Not-so-traditional (but important) properties

Modern security goes well beyond the CIA triad. Notable examples include:

- **Authenticity**: guaranteeing that a piece of information or an entity is indeed what it claims to be.
- **Anonymity**: preventing the identification of the principal behind an action.
- **Isolation**: preventing one component of the system from affecting another.
- **Non-repudiation**: preventing a principal from denying having performed an action.

Remark 1. These properties will be revisited and formalized in later chapters (authentication, anonymity, etc.). The list above is not exhaustive — there are many more complex properties relevant to modern security.

1.3 The security policy

To precisely define a security property, we must specify *what* it applies to. Two notions are introduced for this:

Definition 5 (Assets / objects)**.** Anything with value (data, files, memory, etc.) that needs to be protected.

Definition 6 (Principals / subjects)**.** People, computer programs, services, etc. that interact with the system. A principal **may not** contain the adversary.

Definition 7 (Security policy)**.** A high-level description of the security properties that must hold in the system, in relation to assets and principals.

Property	**General definition**	**Example (principals / assets)**
Confidentiality	Prevention of unauthorized disclosure of information	authorized users may read a file
Integrity	Prevention of unauthorized modification of information	authorized programs may write a file
Availability	Prevention of unauthorized denial of service	authorized services can access a file

Table 1.1: Examples of security properties expressed in terms of principals and assets.

1.4 The adversary and the threat model

Definition 8 (Threat model)**.** Describes the resources available to the adversary and their capabilities (observe, influence, corrupt, …).

A key point of the course: the adversary is **strategic**. This means they will choose *the best possible way* to use their resources to mount an attack that violates the security properties — one should never assume the adversary will act sub-optimally.

Remark 2. Threat modelling is a **very hard task** in practice: underestimating the adversary's capabilities invalidates the security argument, while overestimating them can make the system

impossible or too costly to build.

1.4.1 Key vocabulary to talk about the adversary

Term	Definition	Examples
Threat	The feared event, the goal of the adversary that we don't want to see materialize	A hacker wants to retrieve money by breaking into the bank's system; a student wants to learn my password by looking over my shoulder
Threat model	Technical term defining the adversary's capabilities	The adversary can observe my connection; the adversary can corrupt my machine; the adversary controls a bank employee
Harm	The bad thing that happens when the threat materializes	The adversary steals money; the adversary blocks access to the bank; the adversary learns my password; the adversary reads the message
Vulnerability	A specific weakness that could be exploited by adversaries interested in a variety of different assets	The banking API is not protected; the password appears in plain text on my screen

Table 1.2: Basic vocabulary for describing an adversary.

Remark 3 (Putting it all together)**.** Computer security means guaranteeing that properties (defined by the **security policy**) hold in the presence of an adversary (described by the **threat model**). If the properties hold within that threat model, it means there is no exploitable **vulnerability** that could materialize a **threat**, and hence no **harm** occurs.

1.5 Securing a system: security mechanisms

Definition 9 (Security mechanism)**.** A technical mechanism used to ensure that the security policy is not violated by an adversary within the threat model.

A security mechanism can combine:

- **Software** (programs),
- **Hardware**,
- **Maths** (cryptography),
- **Distributed systems**, **people**, and **procedures**.

Remark 4. Security mechanisms can be *engineered*, just like any other part of a system.

Example 1 — Policy: ensure the log of transactions is not tampered with by a single employee. Mechanism: keep a copy of the log on multiple computers, such that no single employee has access to all of them.

Example 2 — Policy: ensure messages cannot be read by anyone but the sender and the receiver. Mechanism: encrypt the message before sending it.

1.5.1 Composing multiple mechanisms

Real systems are large and often require **several** security mechanisms combined together. There are two fundamentally different ways of composing them, and distinguishing them clearly is crucial:

Definition 10 (Defence in depth)**.** The security policy is maintained **as long as at least one** mechanism remains effective ("OR" composition).

Definition 11 (Weakest link)**.** The security policy is violated **if any one** mechanism fails ("AND" composition).

Remark 5. Important point for exercises: **security does not necessarily increase linearly with the number of mechanisms**. Depending on how mechanisms are composed, adding one can either strengthen the system (defence in depth) or introduce a new point of failure (weakest link) — e.g., a poorly designed mechanism added "in series" can weaken an otherwise solid system.

1.5.2 Humans are part of the system

Humans are an integral part of the system, and are often directly targeted by the adversary:

- **Phishing** attacks and **social engineering**.
- Bad use of passwords: **weak** passwords, passwords **written down**, or passwords **repeated** across services.

Remark 6. The fact that humans are often the "weakest link" does **not** mean the rest of the system should be neglected: all mechanisms must remain robust.

1.6 How do we show systems are secure?

A very important, asymmetric way of reasoning about security:

- **Attacker**: it is enough to find **one single** way to violate **one single** security property (within the threat model) to "win".
- **Defender**: must show that **no** adversary strategy can violate the security policy.

This asymmetry is what makes security hard: the defender must cover *every* case, while the attacker only needs *one* success.

> "One of the major problems right now is script kiddies. These are people who just download open source tools that are meant for good, and they point them at whatever they want, press 'Go,' and it fires a suite of exploits at a system hoping one of them will work." — Richard Moore, Security Specialist (IBM)

Definition 12 (A "secure" system)**.** A system is said to be **secure** if an adversary constrained by a specific *threat model* **cannot** violate the security policy.

Remark 7. The security of a system is therefore never absolute: it is always **relative to a given threat model**. A system may be secure against a weak adversary and no longer secure against a stronger one.

Definition 13 (Security argument)**.** A rigorous argument (verbal or mathematical) that the security mechanisms in place are indeed effective in maintaining the security policy, subject to the

assumptions of the threat model.

Remark 8 (For a threat model to be useful)**.** The model **must constrain** the adversary — otherwise no security argument is possible (an adversary with unlimited capabilities can always break the system).

Remark 9. Security arguments for the **composition** of several mechanisms are particularly hard to get right (cf. *defence in depth* vs *weakest link* above).

1.7 Security engineering

Designing a secure system typically follows three steps:

1. **High-level specification**
   - Define the architecture of the system (e.g., a high-level block diagram).
   - Define the security policy (principals, assets, security properties).
   - Define the threat model.
2. **Security design**
   - Select / design security mechanisms.
   - State the security argument: which controls maintain which properties?
3. **Secure implementation**
   - Implement the mechanisms.
   - Ensure they conform to the design model.
   - Security testing.

Remark 10. Following this order (specify *before* implementing) is essential: a mechanism added without a clear threat model and security argument risks protecting against nothing precise, or creating a false sense of security.

1.8 Summary

- Security problems always involve an **adversary**.
- The adversary is **strategic**: they will take the most damaging approach possible.
- The adversary's **capabilities** define a **threat model**.
- **Security mechanisms** aim at fulfilling a **security policy** within a given threat model.
- Showing that a system is secure requires providing a **security argument**.
- **Composition** of mechanisms is not trivial (*defence in depth* vs *weakest link*).
- Security is always **relative** to the adversary considered.

Chapter 2
Principles of Computer Security

2.1 Motivation

Since we cannot, in practice, prove that a system is free of flaws, the field of computer security relies instead on a set of **design principles**: rules of thumb that, when followed, tend to reduce both the number and the severity of security flaws.

Remark 11 (Historical reference)**.** The principles presented in this chapter were introduced by J. Saltzer and M. Schroeder in *The Protection of Information in Computer Systems*, Proceedings of the IEEE (1975). Despite their age, they remain the foundation of modern security engineering practice: for example, they were used in 2018 by N. Papernot (Google Brain) to structure the threat model of machine learning systems in *A Marauder's Map of Security and Privacy in Machine Learning*.

There are **8** core principles from Saltzer and Schroeder, plus **2** extra principles borrowed from physical security that are also commonly used in computer security.

Remark 12. These principles are **not** a blind checklist to tick off. They are tools to help weigh design decisions and trade-offs. Most real-world security failures come from a principle being improperly evaluated or ignored, rather than from a fundamentally new kind of attack.

2.1.1 Motivating example: the Heartbleed bug

[Heartbleed, CVE-2014-0160] In 2014, a critical vulnerability was found in OpenSSL, the library used to encrypt HTTPS traffic.

- The *Heartbeat* extension of TLS lets a client send a small piece of data and ask the server to echo it back, simply to check that the connection is still alive.
- The client also sends the *claimed length* of the data it is sending.
- **Bug**: the server never checked that the claimed length matched the actual length of the data received. An attacker could send 1 byte while claiming to send 64 000 bytes; the server would reply with the attacker's byte *plus* the next 63 999 bytes of whatever happened to sit in its private memory (potentially including private keys, passwords, or other users' data).
- The missing fix was a single bounds check: `if (request_size < actual_size) { error; }`.

Two structural reasons let this bug survive undetected for two years:

1. **Feature complexity**: the Heartbeat extension was a non-essential feature bolted onto TLS, creating a new attack surface that did not need to exist for the core function of TLS.
2. **Codebase complexity**: OpenSSL is a huge codebase (hundreds of thousands of lines of C and Perl), which makes manual audit extremely difficult.

This example already illustrates the first, and arguably most fundamental, principle: **keep it simple**.

2.2 The eight core principles

2.2.1 1 – Economy of mechanism

Definition 14 (Economy of mechanism)**.** Keep the design of a security mechanism (and its implementation) **as simple and small as possible**.

Why? A security mechanism must be easy to **audit and verify**. Note that *operational testing is not appropriate to evaluate security*: passing functional tests only shows the system behaves correctly on the inputs tested, not that no adversarial input can break it (although penetration testing remains valuable as one tool among others).

Definition 15 (Trusted Computing Base (TCB))**.** Every component of the system (hardware, firmware, software) on which the security policy relies.

The TCB is the only thing that must be *trusted* to operate correctly for the security policy to hold. This gives the only proper technical use of the verb "to trust" in security engineering: *"X trusts Y will do Z."*

Remark 13. If something goes wrong **within** the TCB, the security policy may be violated. If something goes wrong **outside** the TCB, the security policy should still hold. The TCB must therefore be kept as small as possible, both to ease verification (economy of mechanism) and to reduce the attack surface.

2.2.2 2 – Fail-safe defaults

Definition 16 (Fail-safe defaults)**.** Base access decisions on explicit **permission** rather than on **exclusion**.

In other words: if a decision procedure fails, is uncertain, or an error occurs, the system should default to the **safe** state (deny access), not attempt to "fix" the situation on the fly. Concretely, this means using a **whitelist** approach rather than a **blacklist** approach: a lack of permission is easy to detect and does not require anticipating every possible way something could go wrong.

- **Security door**: if no permission was granted, the door does not open.
- **Form input**: if a user has no permission to write into field X, the system does not silently write elsewhere; it refuses.

[MongoDB defaults] For a while, MongoDB's out-of-the-box installation had authentication **disabled by default** and bound to **all** network interfaces (`0.0.0.0`) instead of only the local loop-back (`127.0.0.1`). Attackers scanned the internet for open instances, connected without a password, stole or encrypted the data, and left ransom notes. This is a textbook violation of fail-safe defaults: the default configuration favored ease of use over safety, and many administrators never changed it.

Remark 14 (Ransomware)**.** Ransomware denies the victim access to their own data (typically by encrypting it) and demands payment for the decryption key. The very first documented ransomware dates back to 1989 (the "AIDS Information" floppy disk attack); the phenomenon exploded with the arrival of cryptocurrencies, which make anonymous payment easy. Many ransomware epidemics (e.g., around 2017) directly exploited unsafe defaults such as the MongoDB case above.

2.2.3 3 – Complete mediation

Definition 17 (Complete mediation)**. Every** access to **every** object must be checked for authority.

This principle is usually phrased in terms of a **reference monitor**: a component that mediates *all* actions of subjects on objects, checks them against the security policy, and may log them to an audit log.

[Streaming service authorization] Consider a streaming platform where users can subscribe to a "Gold" tier granting access to exclusive content.

- **Naive (fast) implementation**: the user's Gold status is checked only **once**, at login. The server then issues a session cookie such as `{'user': 'Alice', 'status': 'paid'}`, and every subsequent request only reads this cookie.
- **Attack**: Alice subscribes for one day, logs in (cookie says `paid`), then immediately unsubscribes. The database is correctly updated to `unpaid`, but her cookie is not re-checked. Alice can keep watching premium content until her session expires.

This is a direct violation of complete mediation: not *every* access to the object (the video) was checked against the current policy.

Remark 15 (Why complete mediation is hard to implement)**.**

- **Performance**: checking every single access can be very slow; there is an inherent trade-off between the time spent checking and the time spent actually using the resource.
- **Modern distributed systems**: a reference monitor can only mediate what it *sees* — if part of the system bypasses it (as in the caching example above), mediation is incomplete.

2.2.4 4 – Open design

Definition 18 (Open design)**.** The design of a security mechanism should **not** need to be secret.

This principle has a long history predating computer security:

- **Kerckhoffs** (*La Cryptographie Militaire*, 1883): "The design of a system should not require secrecy."
- **Shannon** (*Communication Theory of Secrecy Systems*, 1949): "The enemy knows the system"; one should design systems assuming the adversary will immediately gain full familiarity with them.
- **Baran** (*Security, secrecy, and tamper-free considerations*, 1964): the "Paradox of the Secrecy About Secrecy" — without the freedom to expose a design to widespread scrutiny by people with diverse expertise, one increases the risk that significant weaknesses go unnoticed.

Remark 16. Open design does **not** mean nothing is kept secret: it means the *algorithm* is public, while only well-defined, easily replaceable **key material** is kept secret. For example:

- **Cryptography**: only the key is kept secret, not the algorithm.
- **Authentication**: only the password is kept secret, not the authentication protocol.
- **Obfuscation**: only the random noise used is kept secret, not the obfuscation method.

Open design leads to better and easier auditing:

> **Linus' law** (E. Raymond, *The Cathedral and the Bazaar*, 1997): "Given enough eyeballs, all bugs are shallow."

This is more of a mantra than a guarantee: it requires genuinely many competent eyeballs, since the density of defects in real software is high. Relying on secrecy instead is unrealistic in practice and is a common way to end up with a bad threat model (*security through obscurity*). A famous failure of

closed design is GSM encryption, whose secret algorithms were reverse-engineered and found to be weak.

[Deep Root Analytics leak, 2017] The personal data (names, addresses, birthdates, political profiling) of 198 million US voters, compiled by data firm Deep Root Analytics, was exposed online in a publicly accessible server with **no password and no authentication**. The only "protection" was that the URL was semi-secret. A security researcher found the URL and disclosed the issue; the 1.1 TB database had effectively been open to anyone on the internet. This is a direct case of security through obscurity failing.

[Scanning the whole internet is cheap] There are only $2^{32} \approx 4$ billion IPv4 addresses. As early as 2013, it was possible to scan every IPv4 address on a given port in about 3 minutes. Consequently, picking a "random" or non-standard port for a service does not hide it from a motivated adversary — it only adds negligible cost.

2.2.5 5 – Separation of privilege

Definition 19 (Separation of privilege)**.** No single accident, deception, or breach of trust should be sufficient to compromise the protected information; **multiple, independent conditions** should be required to perform a sensitive action.

Remark 17. A **privilege** is the ability granted to a user to perform an action with security consequences (e.g., create a file, access a device, write to a network socket).

Requiring two keys to open a safe, or two independent factors to authenticate, are classic instances of separation of privilege. This comes with trade-offs: **availability** (what if one key-holder is unavailable?), **responsibility** (who is accountable?), and added **complexity**.

[Corporate payment system] Consider a business payment system with principals *Employees* (submit orders), *Vendors*, and *Managers* (who can approve new vendors and approve orders, and inherit Employee rights). The security goal is that an attacker must not be able to steal money, under a threat model where the adversary can corrupt **at most one** principal.

If a single corrupted employee can (1) create a fake vendor, (2) submit a fake order for that vendor, and (3) approve the payment themselves, the entire attack can be carried out by one corrupted principal — there is no separation of privilege.

Fix: split the roles so that no single role can complete the whole attack chain:

- *Procurement* role: can only create/approve new vendors, cannot approve payments.
- *Finance Manager* role: can only approve payments for vendors already registered, cannot create new vendors.

Remark 18. Separation of privilege is reminiscent of the "separation of powers" / "checks and balances" principle in political science.

2.2.6 6 – Least privilege

Definition 20 (Least privilege)**.** Every program and every user of the system should operate using the **least** set of privileges necessary to complete the job.

Concretely: rights should be granted **as needed** and discarded **after use**. This limits damage control and minimizes high-privilege actions and interactions — it is the security analogue of the "need-to-know" principle.

Guest accounts at EPFL (limited-duration, limited-scope access), and the *data minimization* principle in data protection law (collect and retain only the data strictly necessary for the stated purpose), are both direct applications of least privilege.

2.2.7 7 – Least common mechanism

Definition 21 (Least common mechanism)**.** Minimize the amount of mechanism common to, and depended on by, more than one user.

Remark 19. This is closely related to economy of mechanism, but focuses specifically on **shared** mechanisms: "every shared mechanism represents a potential information path between users, and must be designed with great care so that it does not unintentionally compromise security."

- At the **design** level, interactions between users through a shared mechanism make the security design harder to validate.
- At the **implementation** level, interactions may lead to unintentional information leaks through *unintended channels*, e.g., a shared `/tmp` directory or a shared cache.

This principle concerns *mechanisms*, not *code*: reusable, well-tested code is generally good for security. The issue is the sharing (lack of isolation) of *data and state*, not code reuse itself.

Remark 20 (Reminder: symbolic links)**.** A symbolic link ("symlink") is a shortcut file: when the operating system accesses it, it is automatically redirected to the real file or directory it points to.

[Violation via a shared `/tmp` directory] Consider three principals: (1) a root/admin program that predictably writes a log file with `echo "Service is starting..." > /tmp/starting.log`; (2) the root/admin logging subsystem; (3) an unprivileged user who can also write to `/tmp`. The security goal is to preserve the integrity of the logs written by the logging subsystem, under a threat model where the adversary can corrupt principal (3).

Attack: the unprivileged user creates, ahead of time, a symlink `ln -s /var/log/dmesg /tmp/starting.log`. When the root program later writes to `/tmp/starting.log`, it actually (over)writes `/var/log/dmesg`, an unrelated privileged log file.

This works because `/tmp` is a mechanism **shared** between low-privilege and high-privilege processes: exactly the setting least common mechanism warns against. (Modern Linux systems mitigate this specific `/tmp` case, but the same attack works if the shared, writable directory is elsewhere, e.g., under `/home/user`.)

2.2.8 8 – Psychological acceptability

Definition 22 (Psychological acceptability)**.** The human interface must be designed for ease of use, so that users **routinely and automatically** apply the protection mechanisms correctly.

Three consequences:

- Security mechanisms should **not** make a resource harder to access than if the mechanism were absent (hide complexity).
- The (honest) user's **mental model** of the system must match the actual security policy and mechanisms.
- **Cultural acceptability**: not every mechanism is acceptable everywhere, e.g., face-recognition authentication is unsuitable in cultures where faces are covered, and requiring a register of everyone sleeping in a dorm raises different acceptability concerns depending on context.

[Amazon S3 permission confusion] Amazon S3 ("Simple Storage Service") stores data as objects inside containers called *buckets*. Psychological acceptability is not only about end users: system administrators configuring a bucket are also "users" of a UI, and that UI can mislead them.

S3's permission UI historically offered an option to grant access to "Any Authenticated AWS User." Administrators commonly read this as "any authenticated user of *my* organization/account," while it actually meant "any person on Earth with *any* AWS account," which is free to create. This mismatch between the mental model and the actual policy caused several major data leaks starting in 2017. Amazon later (2023) introduced an additional layer of fail-safe defaults, blocking public access by default.

Remark 21 (Safer defaults are not free)**.** Tightening defaults for security (e.g., Amazon's 2023 change) can break workflows that legitimately relied on the old, more permissive behavior, generating user frustration. This illustrates that fail-safe defaults and psychological acceptability can be in tension, and trade-offs must be weighed explicitly.

2.3 Two extra principles (from physical security)

2.3.1 9 – Work factor

Definition 23 (Work factor)**.** Compare the **cost** of circumventing a security mechanism with the **resources** available to a potential attacker.

This principle helps refine the threat model: a mechanism need not resist an unlimited adversary, only one bounded by the threat model. In practice, quantifying the work factor is difficult: how does one price the cost of bribing/compromising an insider, the cost of finding a given bug, or the monetary value an attacker could extract?

2.3.2 10 – Compromise recording

Definition 24 (Compromise recording)**.** Reliably record that a compromise has occurred, in place of (or in addition to) more elaborate mechanisms that try to completely prevent loss.

Keeping tamper-evident logs can enable recovery of integrity even when prevention fails. However, logging is not a silver bullet:

- Logging does not guarantee the compromise is detected.
- If confidentiality mechanisms were the ones compromised, logging does not help recover what was already leaked.
- Logs themselves must maintain integrity (what stops the log from being tampered with?).
- Logs can themselves become a vulnerability, e.g., a privacy risk if they contain sensitive data.
- Logging the logs raises its own availability concerns.

2.4 Applying the principles: a machine learning example

As a sign of how durable these 1975 principles are, they are routinely reused to structure entirely new domains. For instance, N. Papernot's 2018 threat-modeling of machine learning security maps directly onto several of them:

- **Least privilege**: let a model learn as little as possible, so that sensitive information cannot later be extracted from it.
- **Least common mechanism**: obtain labelled samples from diverse, independent origins rather than a single shared source.
- **Psychological acceptability**: users must be able to understand *why* a model classifies or misclassifies a given input.
- **Work factor**: the cost of an attack (e.g., number of queries to an API) determines its practical relevance.
- **Compromise recording**: ideally, one would like to log all internal steps of the algorithm to detect compromise.

2.5 Summary

> "Since no one knows how to build a system without flaws, the alternative is to rely on eight design principles, which tend to reduce both the number and the seriousness of any flaws: economy of mechanism, fail-safe defaults, complete mediation, open design, separation of privilege, least privilege, least common mechanism, and psychological acceptability." — Saltzer and Schroeder, 1975

- **Economy of mechanism**: keep the design (and implementation) as simple and small as possible; keep the TCB small.
- **Fail-safe defaults**: base access on permission (whitelist), not exclusion; on failure, default to the safe state.
- **Complete mediation**: verify *every* access to *every* object, every time.
- **Open design**: the design should not need to be secret; only well-defined key material should be.
- **Separation of privilege**: no single compromised principal should be able to complete a sensitive action alone.
- **Least privilege**: operate with the minimum privileges necessary, granted as needed and revoked after use.
- **Least common mechanism**: minimize mechanisms shared between users, to avoid unintended information paths.
- **Psychological acceptability**: the security mechanism must be easy enough to use that honest users apply it correctly and routinely.

- **(Extra) Work factor**: weigh the cost of an attack against the attacker's resources.
- **(Extra) Compromise recording**: when prevention fails, at least reliably detect and record that a compromise occurred.

Remark 22. These principles are guidelines, not proofs. Use them as tools to identify and weigh design trade-offs when building or auditing a security mechanism — not as a checklist that, once ticked, guarantees security.

Lecture 3 — 2026-09-24: Access control basics and DAC

Chapter 3

Access Control

3.1 Introduction to access control

3.1.1 What is access control?

In the physical world, access control is everywhere: a key in a lock, a badge on a reader, a turnstile in a metro station.

The digital world has exactly the same needs, and we will see that the mechanisms are conceptually similar.

Definition 25 (Access control)**.** A security mechanism that ensures that **all accesses and actions on objects by principals are within the security policy**.

An access control system must be able to answer questions such as:

- Can Alice read the file `/users/Bob/readme.txt`?
- Can Bob open a TCP socket to `http://www.abc.com/`?
- Can Charlie write to row 15 of the table `GRADES`?

The answer to each question is binary: either the access is **authorized** (the principal *has permission*), or it is **unauthorized** (*access denied*). The goal is that **only events within the security policy** actually happen.

Remark 23. Recall the vocabulary of Chapter 1: the *principals* (subjects) are the entities acting on the system, the *assets* (objects) are the things to protect, and the *security policy* says which principal may do what on which asset. Access control is the mechanism that *enforces* this policy.

3.1.2 Why is access control so important?

Access control is the **first line of defence**. As a consequence, it is used at *every* layer of a system:

Layer	Examples of what access control protects
Applications	Online social networks, e-mail servers, cloud storage
Middleware	Database management systems (DBMS)
Operating system	Files, directories, ports, …
Hardware	Memory, registers, privilege levels

Table 3.1: Access control is used at every layer of the system stack.

3.1.3 Where does access control fit? Authentication vs. authorization

Access control is usually split into two steps:

1. **Authentication**: the system must *bind the actor to a principal*. A principal is an abstract entity that is authorized to act (a user, a connection, a process, …).
2. **Authorization**: the system must *decide whether the principal is authorized* to perform the requested action on the requested object.

Remark 24. The mechanisms that perform authentication **and** authorization are part of the **Trusted Computing Base** (TCB): if they fail, the security policy may be violated.

3.1.4 Implementing access control: the reference monitor

What NOT to do: "checks soup". A naive implementation sprinkles checks all over the program, each one implementing the policy *in-line*:

```
# some code that needs to access file3.txt
if (action == read) and ((userID == Alice) or (userID == Bob)):
    open(file3.txt, 'r')
elif (action == write) or (userID == Bob):
    open(file3.txt, 'w')
else:
    print("The user does not have access to file3.txt")
```

This is a bad design: the policy is scattered everywhere, which makes it hard to audit, hard to change, and very error-prone. (The example above even contains a bug: the `write` branch uses an `or`, so *anybody* can write the file!) It is the opposite of *economy of mechanism*.

What you SHOULD do: systematic calls to a reference monitor. Throughout the program, add checks that **call a central monitor**. The checks state which authorization is required and provide *evidence* about the principal and the object; the central subsystem (the **reference monitor**) decides whether the check passes.

```
if ( subject.isPermitted("user:delete:jsmith") ) {
    // delete the 'jsmith' user
} else {
    // don't delete 'jsmith'
}
```

The application does not contain the policy: it only asks the monitor.

Remark 25. The reference monitor is how **complete mediation** is put in practice (every access goes through the monitor), and keeping it small and central follows **economy of mechanism**. The slides ask, with a question mark, whether this violates **least common mechanism**: a central monitor *is* a mechanism shared by all users. This is a typical example of *tension between principles*, which must be weighed explicitly rather than ticked off blindly.

3.1.5 Who decides the access policy? DAC vs. MAC

There are two fundamentally different answers to the question "who defines who can access what?".

	Discretionary (DAC)	**Mandatory (MAC)**
Who sets permissions	The **owner** of each object assigns permissions	A **central security policy** assigns permissions
Key idea	Ownership of resources	Organizations with a need for central control
Typical uses	Windows, Linux, macOS; social networks; cloud file sharing	Military (focus on confidentiality); hospitals (confidentiality *and* integrity); banking (focus on integrity)

Table 3.2: Discretionary vs. mandatory access control.

3.2 Discretionary Access Control (DAC)

Definition 26 (Discretionary Access Control)**.** An access control approach in which the **owners of the objects establish the permissions** on them.

3.2.1 Historical motivation: Lampson's protection

DAC policies were conceptualized by B. Lampson in *Protection* (Proc. 5th Princeton Conf. on Information Sciences and Systems, 1971; reprinted in ACM Operating Systems Review 8(1), 1974).

The original motivation for protection mechanisms in computers was to keep *one user's malice or error* from harming another user. Harm can be inflicted by:

a) destroying or modifying another user's data;
b) reading or copying another user's data without permission;
c) degrading the service another user gets (e.g., using up all the disk space or getting more than a fair share of the processing time; an extreme case is crashing the system).

Lampson adds that all these reasons are just as strong if the word "user" is replaced by "*program*": a program (e.g., downloaded malware) can be as dangerous as a malicious user.

These three kinds of harm map directly onto the CIA triad: (b) is a confidentiality violation, (a) an integrity violation, (c) an availability violation.

3.2.2 The Access Control Matrix

In a real system there are many subjects, many objects, and many combinations of permissions between them. How can we reason about them? DAC policies are conceptualized as an **access control matrix**.

Definition 27 (Access control matrix)**.** An **abstract representation of all permitted triplets** (subject, object, access right) within a system. Rows are subjects, columns are objects, and each cell contains the set of operations the subject is allowed to perform on the object.

- **Subjects** (principals): entities within an IT system: a user, a process, a service.
- **Objects** (assets): resources that (some) subject may access or use: a file, a folder, a row in a database, the system's memory, a machine on the network, a printer, a page of a website.
- **Operations**: in the abstract, subjects can *observe* and/or *alter* objects: `read`, `write`, `append`, `execute`.

Let $S = \{\text{Alice}, \text{Bob}\}$, $O = \{\text{file1}, \text{file2}, \text{file3}\}$ and $A = \{\text{read}, \text{write}\}$.

	file1	**file2**	**file3**
Alice	read, write		read
Bob		read, write	read, write

Table 3.3: An access control matrix.

Can Alice read file1? *Yes.* Can Bob write file1? *No* (empty cell: no permission). Can file3 be written by Alice? *No* (she only has `read`).

Remark 26 (Fail-safe defaults)**.** An empty cell means *no permission.* This is the whitelist approach of fail-safe defaults: everything that is not explicitly granted is denied.

Why the matrix is not implemented directly

The access control matrix is an **abstract concept**, not meant for direct implementation. With f files and u users it takes $O(f \cdot u)$ space. For a system with 629 601 files:

Setting	**Matrix size**
1 bit per file, 1 user	$\approx 78\,\text{KB}$
3 bits per file (r/w/x), 1 user	$\approx 236\,\text{KB}$
3 bits per file, 10 users	$\approx 2.36\,\text{MB}$

Beyond raw size, there are more issues:

- The matrix is usually **very sparse**, hence wasteful.
- It is **inefficient to access**: it would have to be kept in fast memory.
- **Extensibility**: what happens when a file or a user is added?

The matrix can be stored in two natural ways: by *column* (permissions attached to objects: **ACLs**) or by *row* (permissions attached to subjects: **capabilities**). Role- and group-based access control are ways of *factoring* the subjects.

3.2.3 Access Control Lists (ACLs)

Definition 28 (Access Control List)**.** An implementation of the access control matrix that **associates permissions to objects**: each object stores the list of (subject, rights) pairs that are allowed to access it. **Blank cells are not stored.**

For the matrix above:

$$\text{file1} : \{(\text{Alice}, \text{read/write})\}$$
$$\text{file2} : \{(\text{Bob}, \text{read/write})\}$$
$$\text{file3} : \{(\text{Alice}, \text{read}), (\text{Bob}, \text{read/write})\}$$

Advantages	**Drawbacks**
Can be stored close to / with the resource	Difficult to check at runtime
Easy to determine *who can access a resource*	Difficult to audit *all the rights of a user*
Easy to *revoke rights by resource*	Difficult to remove all permissions from a user (better remove their authentication!)
	Difficult delegation

Table 3.4: Pros and cons of ACLs.

3.2.4 Role Based Access Control (RBAC)

Systems have **too many subjects**, and they come and go. This leads to large, dynamic ACLs that are painful to maintain. However, subjects are often *similar to each other*: e.g., a doctor has the same privileges as another doctor. RBAC exploits this:

Roles add a level of indirection: when an employee changes job, we change their role assignment rather than editing the ACL of every object.

Problems of RBAC.

1. **Role explosion**: the temptation to create very fine-grained roles removes the benefit of RBAC (we end up with roughly as many roles as subjects).
2. **Limited expressiveness** of simple RBAC: it is hard to implement least privilege precisely, and some roles are *relative*: "Alice's doctor" versus "any doctor".
3. **Separation of duty is difficult**: e.g., "two doctors are needed to authorize a procedure". The RBAC mechanism itself needs to ensure that the two doctors are *distinct*, which a plain role-to-permission mapping cannot express.

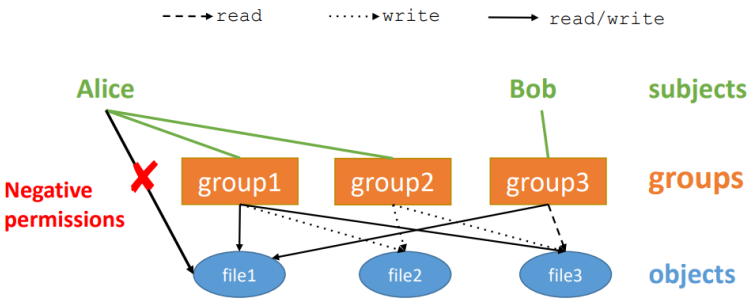
3.2.5 Group Based Access Control

Another way to factor subjects: some permissions are **always needed together** (e.g., access to sockets and to the network interface go hand in hand).

Definition 30 (Group Based Access Control)**.**
1. **Assign permissions** to access objects **to groups**;
2. **assign subjects to groups**;
3. subjects hold **the permissions of all their groups**.

Remark 27 (Groups vs. roles)**.** The difference with RBAC is subtle but important: in group-based access control a subject has the *union* of the permissions of *all* its groups at all times, whereas in RBAC the subject activates *one role at a time* and holds only its permissions, which helps enforce least privilege.

Negative permissions. To express fine-grained policies, group-based systems often add **negative permissions** (explicit denials), e.g., "everyone in group1 may read, except Alice".



Remark 28 (What do we check first?)**. Negative permissions must be checked first.** Otherwise a positive permission obtained through another group could grant the access before the denial is ever seen. Evaluating denials first is also consistent with **fail-safe defaults**. (Windows does exactly this; see below.)

3.2.6 Capabilities

Definition 31 (Capability)**.** An implementation of the access control matrix that **associates permissions to subjects**: each subject holds the list of (object, rights) pairs it is allowed to use. A capability is an *unforgeable* reference to an object that carries the right to use it; holding it is what gives the authority.

For the same matrix:

$$\text{Alice} : \{(\text{file1}, \text{read/write}), (\text{file3}, \text{read})\}$$
$$\text{Bob} : \{(\text{file2}, \text{read/write}), (\text{file3}, \text{read/write})\}$$

Again, blanks are not stored.

Advantages	Drawbacks
Can be stored *with the subject* (portable!)	*Revoking* the permission on one object is hard
Easy to audit *all the permissions of a subject*	*Transferability*: once a capability is given, how can we prevent sharing?
Delegating is "simple" (hand over the capability)	*Authenticity*: how can we check that a capability is genuine (not forged)?

Table 3.5: Pros and cons of capabilities.

Remark 29 (ACLs vs. capabilities: mirror images)**.** The pros and cons are exactly mirrored. ACLs make *per-object* questions easy (who can access this file? revoke access to it) and *per-subject* questions hard. Capabilities make *per-subject* questions easy (what can Alice do? delegate) and *per-object* questions hard. This mirrors the two ways of storing a sparse matrix: by column or by row.

3.3 Ambient authority and the confused deputy

3.3.1 Ambient authority

Definition 32 (Ambient authority)**.** A subject uses **ambient authority** if, for an action to succeed, it **only needs to specify the operation and the names of the involved objects**. The subject holding the authority is *implicit*.

`open("file1", "rw")`: the subject is missing from the call; it is implicitly the *owner of the process*.

- **Advantage**: usability. There is no need to repeat the subject all the time.
- **Drawbacks**: **least privilege becomes harder to enforce** (the process uses *all* the authority of its owner for *every* action), and the program itself **cannot check permissions** on behalf of the party that actually requested the action. This leads to the confused deputy problem.

3.3.2 The confused deputy problem

Definition 33 (Confused deputy problem)**.** A problem of ambient authority: a **privileged program can be tricked into misusing its authority** on behalf of a less privileged requester.

[The pay-per-use compiler] (The classic example, due to N. Hardy.) A compiler is offered as a pay-per-use service. It receives two file names `(input, output)`. It compiles the program `input`, then:

- writes the **usage** (what the user owes) into the file `bill`;
- writes the **errors** into the file `output`.

The compiler's own access rights are those of the service. The ACLs are:

$$\begin{aligned} \texttt{input} &: \{(\text{Alice}, \text{write}), (\text{Compiler}, \text{read})\} \\ \texttt{output} &: \{(\text{Alice}, \text{read}), (\text{Compiler}, \text{read/write})\} \\ \texttt{bill} &: \{(\text{Alice}, \text{read}), (\text{Compiler}, \text{read/write})\} \end{aligned}$$

Can Alice change the bill and avoid paying? Not directly: Alice only has `read` on `bill`. But Alice calls the compiler with arguments `(input, bill)`, i.e., she chooses `bill` as the *output* file. The compiler compiles `input` and writes the errors into "`output`" = `bill`. The compiler is allowed to write `bill`, so the write succeeds and **the bill is corrupted**.

Why does this happen? The compiler *never checks* whether Alice herself would be allowed to write the file she designated as output: it uses its own (ambient) authority to do so. The *designation* of the object (the name chosen by Alice) is separated from the *authority* used to act on it.

How to avoid confused deputies. This is a real problem, because ambient authority is used for convenience in many real systems: OS services, web servers, …Solutions:

1. **Re-implement access control in the privileged process** (e.g. before writing in *output*, the compiler check if output = bill);
2. **let the privileged process check the authorization of Alice** (not only its own);
3. **use capabilities**.

[The compiler with capabilities] The compiler holds a capability to the file `bill` (for its own use). To compile, Alice must *give* the compiler a capability to the debugging file `output`. Alice only holds a read capability on `bill`, so she **cannot hand over a capability for writing on `bill`**. Since the authority travels together with the name of the object, nobody can be confused.

Remark 30 (Exam hints on the confused deputy)**.** Do not just throw the keyword "confused deputy" (or "ambient authority") at a question: link it to the problem at hand. Identify (1) the privileged *deputy*, (2) the less privileged *requester* who controls what the deputy is asked to do, and (3) the action that the requester could not perform alone but that the deputy performs with its own authority. In a past MCQ about a room booking system, the maintenance team (the deputy) cancelling all bookings on a *wrong report* was debated precisely because the question must be read in terms of these three elements.

3.4 Summary of the DAC concepts

- **Discretionary Access Control**: owners establish permissions.
- Conceptualized as an **access control matrix** (abstract; not implemented directly).
- Implemented in two flavours: **access control lists** (permissions associated to objects) and **capabilities** (permissions associated to subjects).
- Roles and groups factor many subjects into a few classes of subjects.
- When relying on access control, always think about **confused deputies**.

3.5 DAC in practice: Linux (UNIX) and Windows

Many of the systems we use every day rely on DAC: social networks, cloud file-sharing systems, and operating systems.

3.5.1 UNIX: principals and groups

- **User identities** (UIDs) and **group identities** (GIDs): originally 16-bit, now 32-bit numbers. Some UIDs are special (e.g., 0 for root, and the special "nobody" user).
- **User information**: each user has their own directory `/home/username`. User accounts are listed in `/etc/passwd`, one line per user with the format

  `username:password:UID:GID:info:home:shell`

- Users belong to **one or more groups**: a *primary* group (given in `/etc/passwd`) and other groups (listed in `/etc/group`).

Security architecture

- **Everything is a file** (regular files, directories, devices, …), so one mechanism protects everything.
- Each user **owns** a set of files.
- Each file has a simple **Access Control List** expressing the policy for that file.
  - The owner can modify the permissions of the file, but **cannot give away ownership**.
  - System files are owned by special users that can perform system operations.
- All processes launched by a user run **with that user's privileges**: this is **ambient authority**!

File ACLs: the 9 permission bits

Each file is assigned an **owner UID** and a **GID**, and has **9 permission bits**: 3 actions (**r**ead, **w**rite, e**x**ecute) for 3 classes of subjects (**owner**, **group**, **other**). Besides these there are 3 special attributes: **suid**, **sgid** and **sticky**.

```
drwxrwxr-x 1 catronco catronco 4096 Sep 16 14:23 exampledir
-rwxrwxrw- 1 catronco catronco 8600 Sep 15 15:20 hello
-rw-rw-rw- 1 catronco catronco  150 Sep 15 15:14 hello.c
-rw-rw--w- 1 catronco catronco   45 Sep 15 15:07 test1.txt
```

Reading a line from left to right: the **type** (`d` for directory, `-` for a regular file), the three **owner** bits, the three **group** bits, the three **other** bits, the number of **links**, the **owner**, the **group**, the **size**, the **last modification** date and the **file name**. For example, `-rw-rw--w-` means: owner may read and write, group may read and write, others may only write.

Bit	**On a file**	**On a directory**
`r` (read)	Read the content	**List** the files in the directory
`w` (write)	Modify the content	**Add** (create) files
`x` (execute)	Run the file as a program	**Traverse** the directory (`cd`, and access what is inside by path)

Table 3.6: Semantics of the permission bits on files vs. directories.

Different semantics for files and directories.

Remark 31 (Directory permissions matter on the whole path)**.** To access `/a/b/c.txt` one needs the `x` (traverse) permission on `/`, `/a` and `/a/b`, on top of the right permission on `c.txt` itself. Creating, deleting or renaming a file is governed by the `w` (and `x`) bits of the *containing directory*, not of the file. This is a classic trap in exercises.

Changing permissions. The owner can change the permissions of a file with `chmod`, either symbolically (`+r`, `-w`, …) or in octal. In octal, read = 4, write = 2, execute = 1 for each of the three triplets, so `666` = `rw-rw-rw-` and `662` = `rw-rw--w-`. The special bits are written with a fourth leading digit: `4000` for suid, `2000` for sgid and `1000` for sticky. For example, `+t` or `1666` sets the sticky bit and `+s` or `4666` sets suid.

How UNIX checks an access

The kernel compares the **UID/GID of the process** trying to perform the action with the **state of the file** (owner, group, mode bits). **The order matters**:

1. If the UID says you are the **owner**: check the **owner** bits.
2. Otherwise, if your **group** is the file's group: check the **group** bits.
3. Otherwise, check the **other** bits.

Remark 32. Only the *first matching class* is used. If you are the owner, the owner bits apply even if the group or other bits are more permissive (so an owner can be *denied* an access that his group is granted). The one exception is the **root user, who is never denied access**.

Super users: root

- Special **root** user account, with **user ID 0**.
- Accesses system files and performs special operations.
- Can access anything: (almost all) security checks are disabled.
- **root is in the TCB!**

Never log in as root (some distributions even assign it no password). Use the `sudo` or `su` commands instead. This is the least privilege principle at work: operate as an ordinary user and elevate only when needed.

Remark 33 (`su` vs. `sudo`)**.** The slide asks about the difference. Briefly: `su` ("switch user") opens a session as another user and requires *that user's* password; `sudo` runs a *single command* with the privileges of another user (root by default) after the *caller* authenticates with their own password, if the policy (`/etc/sudoers`) allows it, and it **logs** each use (compromise recording). In `sudo su catronco`, `su` runs as root, which does not need `catronco`'s password.

Normal users also need to access **system services**, but these services must run with system privileges. This is solved with the **suid/sgid** mechanism.

Special rights: suid and sgid

Definition 34 (Setuid and setgid)**.** The **setuid** and **setgid** bits indicate that an executable file is run not with the privileges of the user who launches it, but with the privileges of the file's owner user (setuid) or owner group (setgid).

This is especially useful for programs that need root privileges while respecting **least privilege**: instead of giving the user root, the user is given a single, well-defined program that runs with elevated privileges. The typical example is changing one's password:

```
ls -l /bin/passwd
-rwsr-xr-x. 1 root root 27768 Aug 20 2020 /bin/passwd
```

The `s` in place of the owner's `x` indicates the setuid bit. Any user can run `passwd`, which runs as root and can therefore write the protected password file, but only in the way the program allows.

Remark 34 (Setuid root programs are dangerous)**.** How do you know that a suid program does what it is meant to do, **and only what it is meant to do**? A setuid-root program runs with full root privileges, so any bug in it can be exploited by the user who launches it. **Setuid root programs are therefore part of the TCB** and must be kept small and audited (economy of mechanism). Imagine what would happen if a general-purpose interpreter such as `python2.7` were

setuid root: anyone could run arbitrary code as root. (See Chen, Wagner, Dean, *Setuid Demystified*, USENIX Security 2002; Kamp and Watson, *Jails: Confining the omnipotent root*, 2000.)

Remark 35 (Exam pitfalls on setuid)**.** Past midterm feedback highlights the following errors:

- **Setuid applies to executables**, i.e., to the *program*, not to a (non-executable) data file.
- Use the **correct notation**: a program owned by `dave` that members of group `managers` may run and others may read is `-rwsr-xr--`.
- Do *not* "fix" a permission problem by making a sensitive file readable/writable by everybody: this removes any notion of access control on it and violates several security principles.

[Exam-style: emergency deletion program] Tropyota wants the CEO Carla (group `managers`) to be able to run an emergency program deleting the secret plans, which only the `designers` may read or modify. The configuration is

```
-rwxr-xr-- dave managers   program_to_delete_plans
-rw-rw---- dave designers  hybris_plans.dwg
```

Does it work? No: Carla can *execute* the program (group `managers` has `r-x`), but with ambient authority the process runs *with Carla's privileges*, and Carla (not in `designers`) has no access to the plans. **Fix**: set the **setuid bit on the program**, giving `-rwsr-xr--`, so that it runs with the privileges of its owner Dave, who can access the plans.

Least privilege? Still not satisfied: *every* manager (not only the CEO) can run the program, and the program runs with all of Dave's privileges, which is more than strictly necessary.

Special rights: the sticky bit

The sticky bit (`chmod +t`) is also called the **restricted deletion bit**.

- **On directories**: it prevents unprivileged users from *removing or renaming* a file in the directory unless they own the file. Example: the `/tmp` folder, which is writable by everyone, but where users can only delete or rename *their own* files.
- **On files**: historically, it kept a program in swap for faster loading. On current Linux, the bit is ignored for files.

Remark 36. Recall the `/tmp` symlink attack of the *least common mechanism* example: `/tmp` is a shared mechanism. The sticky bit limits what users can do to one another's files there, but sharing a writable directory remains risky.

Special user: nobody

`nobody` is a special user (UID -2 in the slides) that **owns no files and belongs to no group of users**. It is a safer user to run code you do not know or trust (particularly obfuscated code), since it **limits the damage** if the code misbehaves or gets compromised: another application of least privilege.

3.5.2 Windows

- **Principals**: users, machines, groups, …
- **Objects**: files, Registry keys, printers, …

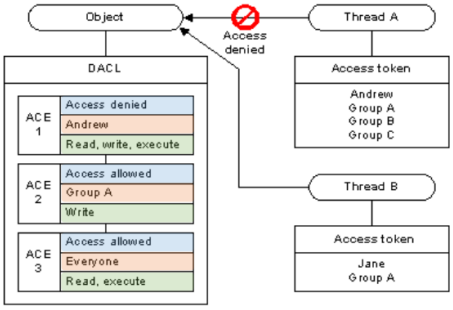
Access control works as follows:

- Each object has a **Discretionary Access Control List (DACL)**.
- Each process (or thread) has an **access token** containing: the *login user account* (the process "runs as" this user), *all the groups* of which the user is a member (recursively!), and *all the privileges* assigned to these groups.
- When a process **creates a handle to an object**, the DACL of the object is compared with the process' access token.

The DACL. A DACL is a list of **Access Control Entries (ACEs)**. Each ACE has:

- a **type**: negative (access denied) or positive (access allowed);
- a **principal**;
- the **permissions**, which are **more fine-grained than in UNIX**;
- flags and others.

List of Access Control Entries (ACEs)



Type: negative / positive

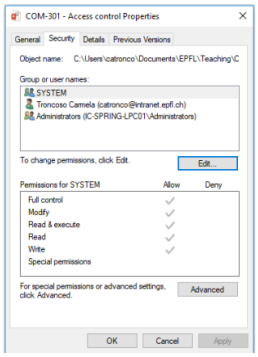
Principal

Permissions: more fine grained than UNIX

+ Flags and others…

Least Privilege by default

Run as administrator



Remark 37 (Why negative first?)**.** ACEs are evaluated *in order*, and **negative ACEs are placed first**: an explicit denial must win over an allowance obtained through another entry (for instance, through a group). The same reasoning as for negative permissions in group-based access control applies.

Least privilege by default. Windows encourages least privilege through "*Run as administrator*": even administrators work with a restricted token and elevate only when needed.

3.5.3 Comparison UNIX vs. Windows

	UNIX	**Windows**
Per-object list	9 permission bits (owner/group/other)	DACL: list of ACEs (type, principal, permissions)
Granularity	3 actions (r, w, x)	More fine-grained permissions
Denials	Implicit (first matching class decides)	Explicit negative ACEs, evaluated first
Subject side	UID, GID(s) of the process	Access token (user, all groups recursively, privileges)
Elevation	root, `sudo`/`su`, suid/sgid	"Run as administrator"

Table 3.7: DAC in UNIX and Windows.

3.6 Summary

- **Access control** ensures that all accesses of principals on objects are within the security policy; it is the **first line of defence** and is used at every layer (application, middleware, OS, hardware).
- It is split into **authentication** (bind the actor to a principal) and **authorization** (decide whether the principal may act); both are in the **TCB**.
- Do not write "checks soup": call a **reference monitor** (complete mediation, economy of mechanism).
- **DAC**: owners decide the permissions (Windows, Linux, macOS, social networks). **MAC**: a central policy decides.
- The **access control matrix** (subjects $\times$ objects $\rightarrow$ rights) is an abstract model, too big and sparse to implement directly.
- **ACLs**: permissions attached to objects (easy to see who accesses a resource, hard to audit a user). **Capabilities**: permissions attached to subjects (easy to audit and delegate, hard to revoke).
- **RBAC** (roles, one active role) and **groups** (union of all groups, negative permissions checked first) reduce the management cost of many subjects. RBAC suffers from role explosion, limited expressiveness and difficult separation of duty.
- **Ambient authority** is convenient but makes least privilege harder and causes the **confused deputy problem**. Solutions: re-check authorization in the privileged process, or use capabilities.
- **UNIX**: UIDs/GIDs, 9 permission bits (owner, group, other), check order owner $\rightarrow$ group $\rightarrow$ other, root never denied, directories have different semantics (list/add/traverse), **suid/sgid** (run with the owner's privileges; dangerous, in the TCB), **sticky bit** (restricted deletion, e.g., `/tmp`), **nobody** (safe user for untrusted code).
- **Windows**: DACL = list of ACEs (negative/positive, principal, fine-grained permissions), compared with the access token of the process; negative entries first.